

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	Date:	Enrolment No: 92410133034

Aim: To study different word embedding for representation of textual data in vectorized form

IDE: Google Colab

Theory:

Word Embedding is an approach for representing words and documents. Word Embedding or Word Vector is a numeric vector input that represents a word in a lower-dimensional space. It allows words with similar meanings to have a similar representation.

Word Embeddings are a method of extracting features out of text so that we can input those features into a machine learning model to work with text data. They try to preserve syntactical and semantic information. The methods such as Bag of Words (BOW), CountVectorizer and TFIDF rely on the word count in a sentence but do not save any syntactical or semantic information. In these algorithms, the size of the vector is the number of elements in the vocabulary. We can get a sparse matrix if most of the elements are zero. Large input vectors will mean a huge number of weights which will result in high computation required for training. Word Embeddings give a solution to these problems.

Need for Word Embedding?

- To reduce dimensionality
- To use a word to predict the words around it.
- Inter-word semantics must be captured.

How are Word Embeddings used?

- They are used as input to machine learning models.
Take the words —> Give their numeric representation —> Use in training or inference.
- To represent or visualize any underlying patterns of usage in the corpus that was used to train them.

1. Bag of Words

Bag of Words (BOW) is an algorithm that counts how many times a word appears in a document. Those word counts allow us to compare documents and gauge their similarities for applications like search, document classification, and topic modeling.

2. Tf-Idf

Tf-Idf is shorthand for term frequency-inverse document frequency. So, two things: term frequency and inverse document frequency. Term frequency (TF) is basically the output of the BoW model. For a specific document, it determines how important a word is by looking at how frequently it appears in the document. Term frequency measures the importance of the word. If a word appears a lot of times, then the word must be important. For example, if our document is “I am a cat lover. I have a cat named Steve. I feed a cat outside my room regularly,” we see that the words with the highest frequency are I, a, and cat. This agrees with our intuition that high term frequency = higher importance.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	Date:	Enrolment No: 92410133034

$TF(t) = (\text{Number of times term } t \text{ appears in a document}) / (\text{Total number of terms in the document})$

IDF used to calculate the weight of rare words across all documents. The words that occur rarely in the corpus have a high IDF score. However, it is known that certain terms, such as “I”, “a” may appear a lot of times but have little importance. Thus we need to weigh down the frequent terms while scaling up the rare ones

$IDF(t) = \log_e(\text{Total number of documents} / \text{Number of documents with term } t \text{ in it})$

Pre Lab Exercise:

1. Perform BoW vectorization of the corpus

Bag of Words (BoW) converts text into numerical form by counting the frequency of each word in the corpus. It creates a vocabulary of all unique words and represents each document as a vector of word counts.

Example:

Sentence → “I love AI”

BoW → {I:1, love:1, AI:1}

2. Perform TF vectorization of the corpus

Term Frequency (TF) measures how frequently a word appears in a document. It is calculated as the ratio of number of times a word appears to total number of words in the document.”

Formula:

$TF = (\text{Number of times word appears}) / (\text{Total words in document})$

It gives importance to frequently occurring words in a document.

Program (Code):

To be attached with

1. Perform the TF-IDF vectorization of the corpus

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	Date:	Enrolment No: 92410133034

```
[5]
✓ Os
▶ from sklearn.feature_extraction.text import TfidfVectorizer

# Sample corpus
corpus = [
    "I love machine learning",
    "Machine learning is interesting",
    "I love AI"
]

# Create TF-IDF model
vectorizer = TfidfVectorizer()

# Fit and transform
tfidf_matrix = vectorizer.fit_transform(corpus)

# Show feature names
print("Features:", vectorizer.get_feature_names_out())

# Show TF-IDF values
print("\nTF-IDF Matrix:\n", tfidf_matrix.toarray())
```

... Features: ['ai' 'interesting' 'is' 'learning' 'love' 'machine']

TF-IDF Matrix:

 [[0. 0. 0. 0.57735027 0.57735027 0.57735027]

 [0. 0.5628291 0.5628291 0.42804604 0. 0.42804604]

 [0.79596054 0. 0. 0. 0.60534851 0.]]

Observation:

TF-IDF vectorization converts text into numerical form by considering both term frequency and inverse document frequency. It gives higher weight to important words and reduces the importance of frequently occurring common words.

Post Lab Exercise:

Take three documents and find similarity using

- BoW vectorization
- TF Vectorization
- TF-IDF Vectorization

Comment over the answer obtained in each of the case.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Artificial Intelligence (01CT0616)	Aim: To study different word embedding for representation of textual data in vectorized form	
Experiment No: 8	Date:	Enrolment No: 92410133034

```
[6]
✓ 0s
▶ from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
  from sklearn.metrics.pairwise import cosine_similarity

# Sample documents
docs = [
    "I love machine learning",
    "Machine learning is amazing",
    "I enjoy AI"
]

# ♦ 1. BoW (CountVectorizer)
bow = CountVectorizer()
bow_matrix = bow.fit_transform(docs)
bow_sim = cosine_similarity(bow_matrix)

print("Bow Similarity:\n", bow_sim)

# ♦ 2. TF (same as BoW normalized)
tf_sim = cosine_similarity(bow_matrix)
print("\nTF Similarity:\n", tf_sim)

# ♦ 3. TF-IDF
tfidf = TfidfVectorizer()
tfidf_matrix = tfidf.fit_transform(docs)
tfidf_sim = cosine_similarity(tfidf_matrix)

print("\nTF-IDF Similarity:\n", tfidf_sim)
```

```
... Bow Similarity:
[[1.         0.57735027 0.         ]
 [0.57735027 1.         0.         ]
 [0.         0.         1.         ]]

TF Similarity:
[[1.         0.57735027 0.         ]
 [0.57735027 1.         0.         ]
 [0.         0.         1.         ]]

TF-IDF Similarity:
[[1.         0.44333251 0.         ]
 [0.44333251 1.         0.         ]
 [0.         0.         1.         ]]
```